Group 5 – Almanac

Addison Broschell, Arianna Wallace, Harveen Harveen, Leo Goebel, and Aithan McIntyre

Decay Signature Comparison

Decay signature	jq – Human Written	Meridian – Agent era
Duplicated logic	Repeated memory initialization in Windows support code. Creates extra maintenance work when changes are needed.	Station and trend reports independently calculate yields. Changes to one report may not carry over to the other.
Documentation drift	Outdated reference-count comment no longer accurately describes every function.	README omits trend-report and JSON functionality, leaving documentation incomplete.
Incomplete abstractions	Certain input errors bypass the shared callback, resulting in inconsistent error handling.	Reports use different forecast-selection rules despite attempts to centralize this logic. This can lead to inconsistent results.

Key findings and Change impact

- Our comparison found that both the human-written jq project and agent-era Meridian exhibit similar signs of software decay.

- Meridian's issues are particularly concerning because duplicated logic and incomplete abstractions can lead to inconsistencies in generated reports.

- One example is duplicated yield-calculation logic in Meridian, where station and trend reports perform similar calculations separately. If one calculation is updated without changing the other, the reports may produce inconsistent results.

- These findings suggest that consolidating shared calculations and improving reporting abstractions could reduce maintenance effort and improve consistency across Meridian's reporting functions.

Tickets

MRD-238: The report writer selects the current forecast per station with `MAX(run_at)` compared as text. A manual backfill written in a different timestamp format sorts wrong, so a report can print an older run as the one "in force".

MRD-250: `reports/selection.current_forecast` orders runs by comparing `run_at` as text (the MRD-238 hazard re-seeded in the new Python selection path). An off-format backfill sorts wrong, so a newer backfill is picked as an older run. Latent on ISO-stamped data.

Both of these tickets address a similar issue based on formatting, leading to the possibility of an older forecast being selected

Impact-change

More than one report depends on the timestamp selection:

- Station reports use latest_forecast_for_station() in db.py.
- Trend reports use current_forecast() in selection.py.
- Both use text-based timestamp comparisons, so both selection paths need to be considered when making changes.

How it shapes our changes

1. Fix both forecast-selection issues
- Update the functions in db.py and selection.py to correctly identify the newest forecast.

2. Consider the related functions
- Check other functions that depend on timestamp selection.
- Make sure station and trend reports select forecasts consistently.

3. Make sure nothing else breaks
- Test with different timestamp formats.
- Compare reports before and after the changes.
- Rerun the existing report tests.

Degenerate Pattern

Pattern: Eliminate Navigation Code

Files: emitters.py, columns.py

Should be removed because emit_json disregards the robust, premade ways, in favour of manual navigation.

```python
# --- JSON ---------------------------------------------------------------
def emit_json(doc: ReportDocument, *, indent: Optional[int] = 2) -> str:
    """Render the report's tables as a machine-readable JSON document. ···
    tables: List[Dict[str, Any]] = []
    for block in doc.blocks:
        if block.columns is None:
            continue
        cs = block.columns
        columns = [
            {
                "key": c.key,
                "title": c.title.strip(),
                "unit": c.unit,
                "kind": c.kind,
            }
            for c in cs.columns
        ]
        rows = [
            {c.key: _json_cell(row.get(c.key)) for c in cs.columns}
            for row in block.rows
        ]
        tables.append(
            {"heading": block.heading, "columns": columns, "rows": rows}
        )

    payload: Dict[str, Any] = {
        "report": {
            "title": doc.running_title,
            "reference": doc.ref_code,
        },
        "tables": tables,
    }
    return json.dumps(payload, ensure_ascii=True, indent=indent) + "\n"
```

THANK YOU FOR LISTENING